

AI-Powered Code Snippet Manager

Efficient, Organized, and Intelligent Coding

Version: 1.0
Prepared By: Allaka Mahesh
Date: July 02, 2026

1. Introduction

1.1 Purpose

This document specifies the software requirements for the AI-Powered Code Snippet Manager. The primary goal is to provide developers with a robust environment to securely save, organize, and retrieve code snippets, augmented by Gemini AI for code optimization and explanation.

1.2 Project Scope

The application is a web-based ecosystem tailored for students and professional engineers. It encompasses the end-to-end lifecycle of a code snippet: from creation and cloud-syncing to public sharing and AI-driven analysis.

2. Objectives

- **Redundancy Reduction:** Minimize time spent rewriting similar code blocks.
- **Intelligent Assistance:** Use Large Language Models to debug and optimize logic on the fly.
- **Accessibility:** Secure cloud storage ensuring snippets are available across all user devices.
- **Collaboration:** Facilitate learning through public sharing mechanisms.

3. Technology Stack

FRONTEND

React.js, Tailwind CSS, Monaco Editor, Axios

BACKEND

Node.js, Express.js, JWT Authentication

DATABASE

MongoDB Atlas, Mongoose ODM

AI & CLOUD

Gemini API, Vercel, Render/Railway

4. Functional Requirements

FR-1: User Authentication

Support for secure registration, login via JWT, and profile management including avatars and password recovery.

FR-2: Smart Dashboard

Centralized view for recent activity, favorite snippets, collection shortcuts, and usage analytics.

FR-3: Monaco Editor Integration

Full IDE-like experience with syntax highlighting, auto-indentation, and multi-language support (JS, Python, C++, etc.).

FR-4: AI Assistant Module

Integration with Gemini to: Explain code, find bugs, optimize performance, and auto-generate documentation/unit tests.

FR-5: Version Control

Automatic immutable history of every snippet edit, allowing users to compare and roll back to previous versions.

5. Database Schema

The system utilizes a relational-style document schema in MongoDB:

ENTITY	KEY ATTRIBUTES
User	_id, name, email, password (hashed), avatar
Snippet	title, code, language, tags[], ownerId, isPublic
Collection	name, ownerId, snippetIds[]
Version	snippetId, codeBlob, timestamp

6. API Documentation (Key Endpoints)

- `POST /api/auth/register` - Account creation
- `GET /api/snippets` - Fetch user-specific library
- `POST /api/ai/optimize` - Trigger AI performance analysis
- `GET /api/share/:link` - Public read-only access

7. Non-Functional Requirements

- **Performance:** Initial load under 2s; UI interactions under 100ms.
- **Security:** AES encryption for sensitive data; Bcrypt for passwords.
- **Availability:** 99.9% uptime target using containerized deployments.

8. Success Criteria

Deployment to production with a fully responsive UI, functional AI integration, and zero critical vulnerabilities during security auditing.